

@intentsolutions/audit-harness: Operator-Grade System Analysis

Generated: 2026-05-20 Version: v1.0.1 (commit 483945a)

1. This System in 5 Minutes

@intentsolutions/audit-harness is a 1,800-line polyglot test-enforcement toolkit shipped as three OS packages — @intentsolutions/audit-harness on npm, intent-audit-harness on PyPI, and intent-audit-harness on crates.io — plus a vendor-by-curl installer for every language that does not have a package manager wired up. Its purpose is narrow and deliberate: enforce that *policy changes to a repository's test posture are conscious, not silent*. Every consumer repo installs the harness as a dev-dependency, hashes its engineer-owned policy artifacts into a `.harness-hash` manifest, and pipes its diffs through `audit-harness escape-scan --staged` at pre-commit and CI time. Any AI agent or contributor that tries to lower a coverage floor, delete a test file, downgrade an architecture rule, or skip a mutation marker gets caught at the gate with a deterministic regex grammar (REFUSE / CHALLENGE / FLAG) — no LLM in the loop, no judgment call, no plausible deniability.

Who uses it: every repo in the Intent Eval Platform umbrella (`intent-eval-lab`, `intent-eval-core`, `j-rig-skill-binary-eval`, `intent-rollout-gate`) plus an unknown but non-zero number of external consumers downloading from the public npm tarball (~1,140 historical downloads per the Phase B note in `CLAUDE.md:40`). The harness is the L1+L3 enforcement substrate underneath the larger 7-layer testing taxonomy that the `audit-tests` and `implement-tests` Claude Code skills install. Per the corporate testing SOP (umbrella `CLAUDE.md`, “Intent Solutions Testing SOP”), the harness must travel with the code — pre-commit hooks and CI reference `pnpm exec audit-harness ...` or `scripts/audit-harness ...` in the *target* repo, never the user's `~/.claude/` global skill directory. This makes fresh clones reproduce every gate.

How it works mechanically: `bin/audit-harness.js` (`bin/audit-harness.js:15-25`) is a thin Node dispatcher — a 105-line `child_process.spawn` shim that maps eight subcommand names to either a `scripts/*.sh` (bash) or `scripts/*.py` (python3) implementation. The shell and python scripts are the *real* product. They read repo-local policy (typically `tests/TESTING.md` for floors and `.dependency-cruiser.cjs`, `.importlinter`, `stryker.conf.json`, etc. for architecture rules), pin those files' SHA-256 hashes into `.harness-hash`, and refuse diffs that mutate them without a fresh `audit-harness init`. The v0.3.0 Milestone-2 work added `--json` to every gate and an `emit-evidence` subcommand (`scripts/emit-evidence.sh:55-56`) that wraps each gate's JSON output in an in-toto Statement v1 with `predicateType: https://evals.intentsolutions.io/gate-result/v1`. This is the convergence layer with `intent-eval-lab` — every gate result becomes a row in an Evidence Bundle that downstream tools (`j-rig`, `intent-rollout-gate`) consume.

Current state: at v1.0.1 the package is operationally healthy. CI passes on Node 18/20/22 (`.github/workflows/ci.yml:21-22`); a 4-section, 11-check backward-compat regression suite (`tests/regression/run-regression.sh`) protects the `--json` + `emit-evidence` additions from drift; SemVer commitments are codified (`SEMVER.md:30-55`); the license is Apache 2.0 with a NOTICE file shipped in the tarball (the v1.0.1 fix). The repo has 4 git tags shipped, 19 commits on `main`, and zero runtime dependencies in the Node dispatcher. The harness is *self-aware enough* to be tested against synthetic diffs in CI (`.github/workflows/ci.yml:53-78`) but is *not yet*

self-pinning — the `verify` and `list` self-check steps tolerate exit-3 (“no manifest”) because the repo does not yet initialize a manifest against its own scripts.

The biggest risk is supply-chain attribution. The harness sits in CI as the *last* gate before a merge — a forged `verify` output, a tampered tarball, or a downgraded predicate URI could silently disable test enforcement across every consumer repo simultaneously. The current mitigations (dependabot weekly, CODEOWNERS lock on `/scripts` and `/bin`, SemVer commit doc, security@ contact) are good policy hygiene but lack technical depth: no cosign provenance on the npm publish workflow, no Rekor entry for tagged releases, and the `emit-evidence --sign` path is gated behind an unverified DNSSEC + CAA precondition on `evals.intentsolutions.io` (see `scripts/emit-evidence.sh:39-43` and the CISO binding referenced there). Secondary risks: a three-way version drift between `package.json` (1.0.1), `python/pyproject.toml` (0.1.0), `rust/Cargo.toml` (0.1.0), and `version.txt` (0.2.0) means a consumer asking “what version am I running?” gets four different answers depending on install path; and the bash + python polyglot pattern is portable but increases the surface area of `set -euo pipefail` discipline that has historically been the source of subtle non-determinism in the test corpus.

2. Executive Summary

What It Does

The harness implements eight deterministic test-quality gates accessible through a single CLI surface: `verify`, `init`, `list`, `escape-scan`, `arch`, `bias`, `gherkin-lint`, `crap`, and `emit-evidence`. Each gate is a standalone bash or python script in `scripts/`. The Node dispatcher at `bin/audit-harness.js:15-25` maps the user-facing command to the script and shells out via `child_process.spawn` with `stdio: 'inherit'`. Every gate accepts a `--json` flag (v0.3.0+) that emits a partial gate-result envelope to stdout while preserving the existing human-readable text on stderr — the stream split is the entire backward-compat strategy. Run by themselves they FAIL (exit non-zero) or PASS the engineer locally; pipe their `--json` output through `emit-evidence` and they become signed in-toto Statements with `predicateType https://evals.intentsolutions.io/gate-result/v1` suitable for transparency-log push.

Implementation status: stable. All eight gates are implemented and exercised by CI on three Node versions. The `--json + emit-evidence` Milestone 2 work landed in v0.3.0; the license relicense to Apache 2.0 landed in v1.0.0; the tarball-NOTICE fix landed in v1.0.1. The 1,808 lines of code (`scripts/* + bin/audit-harness.js + tests/regression/run-regression.sh` per `wc -l`) are tracked, reviewed under CODEOWNERS, and packaged for three ecosystems. Polyglot wrappers (`python/`, `rust/`) re-implement the dispatcher in their respective host languages and re-bundle the *same* scripts via package data (Python) or `include_bytes!` (Rust). The convergence-layer feature — Evidence Bundle emission — is implemented and tested but BLOCKED from production signing until the DNSSEC + CAA precondition on `evals.intentsolutions.io` is met (per `scripts/emit-evidence.sh:39-43` and ISEDC Session 1 binding referenced in `CHANGELOG.md:55-58`).

Technology foundation: Node 18+ for the dispatcher; bash for six of seven scripts; python3 for `crap-score.py` and `emit-evidence.sh`'s JSON marshaling sub-call (`scripts/emit-evidence.sh:123-169`); cosign (operator-installed) for the optional sign step; jsonschema (Python) for CI-time schema validation of emitted envelopes (`.github/workflows/ci.yml:113`). Zero npm runtime dependencies in the Node package — `package.json:39-49` lists only a `test` script and a

`prepublishOnly` smoke check; the engines field pins Node 18+. The architecture-check dispatcher (`scripts/arch-check.sh`) shells out to the *consumer repo*'s installed checker (dependency-cruiser, import-linter, deptrac, arch-go, or ArchUnit via gradle) but does not install any of them.

Key risks: (1) supply-chain integrity — the npm publish workflow does not yet emit sigstore provenance or push a Rekor entry for tagged releases, so a tampered tarball cannot be deterministically rejected by consumers; (2) version drift across the four installation surfaces — `package.json` (1.0.1), `version.txt` (0.2.0), `python/pyproject.toml` (0.1.0), `rust/Cargo.toml` (0.1.0) — currently disagree, which means `--version` answers differ by install path; (3) the harness does not yet self-pin — CI tolerates exit-3 (“no manifest”) on `verify` and `list` (`.github/workflows/ci.yml:36-51`), so a forged change to a script in this repo would not be caught by this repo's own harness; (4) the `emit-evidence --sign --rekor-url` codepath is operator discipline, not enforced by the script — it will happily push to Rekor against a predicate URI whose DNS posture has not been verified.

Operational Status

Table 1.

Environment	Status	Uptime Target	Release Cadence	Last Deploy
Production (npm public)	Live as <code>@intentsolutions/bin/audit-harness@0.1.0</code>	N/A (package, not intent-audit-harness@0.1.0)	On-demand (4 releases since 2026-04-21)	2026-05-20 (v1.0.1)
Production (PyPI)	Live as <code>intent-audit-harness@0.1.0</code> (out-of-sync)	N/A	One release shipped	2026-04 (per <code>python/pyproject.toml</code>)
Production (crates.io)	Live as <code>intent-audit-harness@0.1.0</code> (out-of-sync)	N/A	One release shipped	2026-04 (per <code>rust/Cargo.toml</code>)
Local dev	Bash + Node 18+, smoke via <code>node bin/audit-harness.js --version</code>	N/A	N/A	N/A

Environment	Status	Uptime Target	Release Cadence	Last Deploy
CI (GitHub Actions)	Three jobs: self-check (3-Node matrix), shellcheck (advisory), python-syntax (gating), regression (gating)	Green on main	Per push + nightly cron at 06:00 UTC	per commit

Technology Stack

Table 2.

Category	Technology	Version	Purpose
Dispatcher language	Node.js	>=18 (engines field)	Cross-platform CLI shell, <code>child_process.spawn</code> to scripts
Script language (majority)	Bash	5+ (assumed; no explicit pin)	Six of seven gate scripts (<code>harness-hash</code> , <code>escape-scan</code> , <code>arch-check</code> , <code>bias-count</code> , <code>gherkin-lint</code> , <code>emit-evidence</code>)
Script language (CRAP)	Python	3.10+ per <code>CONTRIBUTING.md:12</code> , but <code>python/pyproject.toml:18</code> says >=3.8; CI uses 3.12	<code>crap-score.py</code> + JSON marshaling inside <code>emit-evidence.sh</code>

Category	Technology	Version	Purpose
Optional signing	cosign	Latest (operator-installed)	<code>emit-evidence</code> <code>--sign</code> — Fulcio OIDC keyless or <code>--key</code> keyref
Optional transparency log	Rekor (sigstore)	sigstore default	<code>--rekor-url</code> push of signed DSSE envelope
CI orchestration	GitHub Actions	<code>actions/checkout@v6</code> , <code>github/workflows/ci.yml</code> <code>setup-node@v6</code> , <code>setup-python@v6</code>	
Package ecosystems	npm + PyPI (hatchling) + crates.io (cargo)	Each independent	Triple-publish for cross-ecosystem reach
Schema validation	<code>jsonschema</code> (Python)	CI-installed only	Regression-suite Section 3 validates emitted envelopes against <code>gate-result.schema.json</code>
Architecture checkers (dispatched, not bundled)	<code>dependency-cruiser</code> / <code>import-linter</code> / <code>deptrac</code> / <code>arch-go</code> / <code>ArchUnit</code> (Gradle)	Consumer-installed	<code>scripts/arch-check.sh</code> detects which is configured and invokes it
Complexity scorers (dispatched)	<code>radon</code> (Py) / <code>cr</code> / <code>complexity-report</code> (JS) / <code>gocyclo</code> (Go) / <code>rust-code-analysis-cli</code> (Rust)	Consumer-installed	<code>scripts/crap-score.py:99-279</code>

3. Architecture Stack (Detailed)

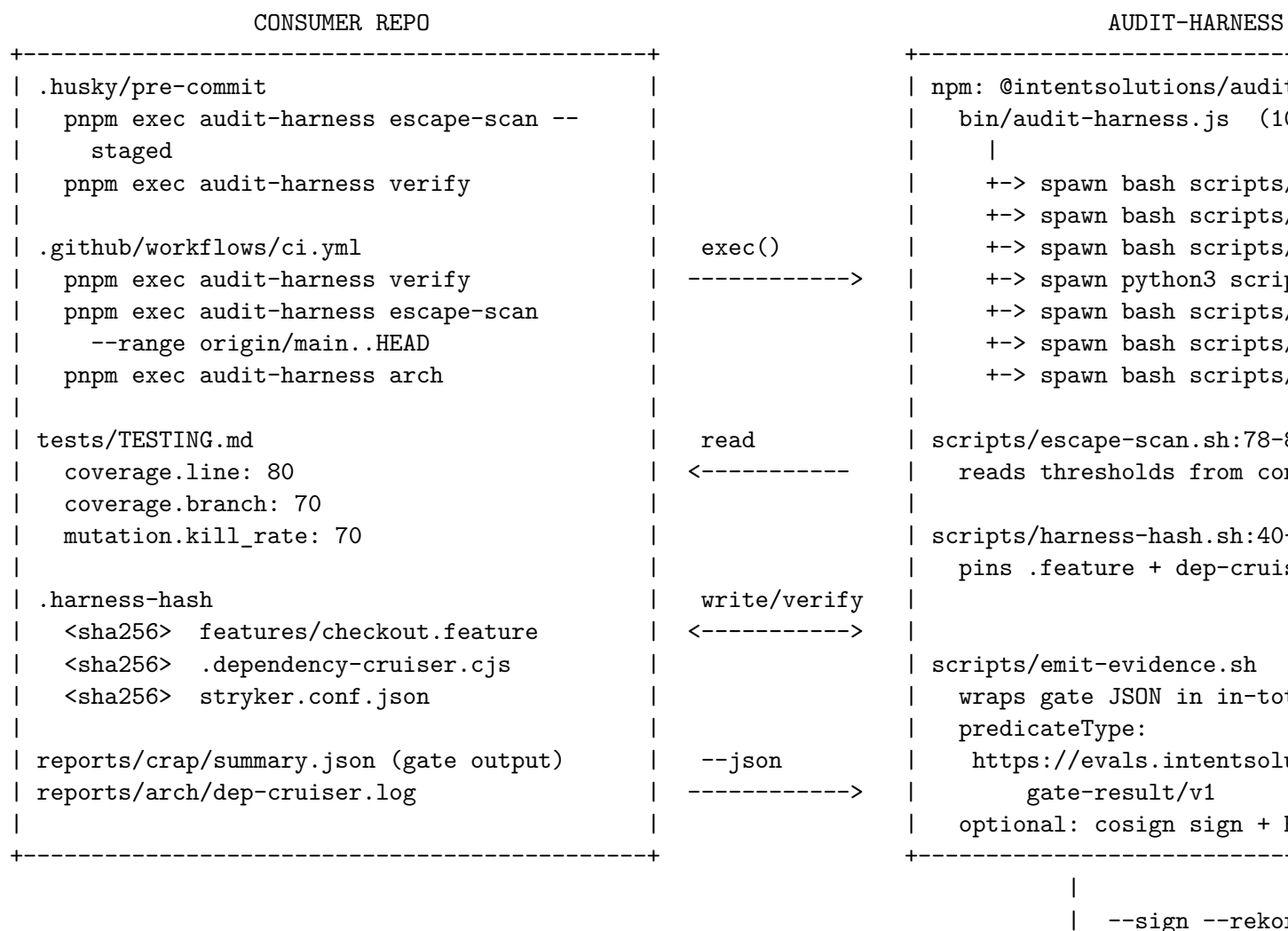
Table 3.

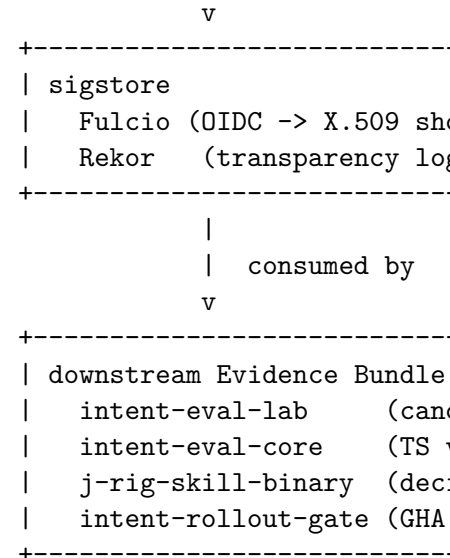
Layer	Technology	Version	Purpose	Why This
User-facing CLI	Node dispatcher (<code>bin/audit-harness.js</code>) <code>deps</code>	105 LOC, no	Single entry point, cross-platform-ish stdio piping	Node has the broadest “is already installed in this engineer’s CI image” footprint; alternatives (Python entry point, Rust binary) exist as polyglot wrappers but Node is the canonical dispatcher per <code>CLAUDE.md:11</code> “Scripts are the source of truth. The Node CLI is a thin dispatcher”
Gate implementation (shell)	bash 5 idioms (<code>set -euo pipefail</code> , <code>[[]]</code> , process substitution, <code>mapfile</code> patterns)	6 scripts, ~1,500 LOC	The actual gate logic	“Battle-tested, language-portable, minimal dependencies” per <code>CHANGELOG.md:93</code> ; ports to TS were considered and rejected unless a “concrete reason (cross-platform Windows bug)” exists per <code>CLAUDE.md:11</code>
Gate implementation (python)	<code>crap-score.py</code> , <code>emit-evidence.sh</code> ’s embedded <code>python3 -<<PY</code> block	~430 LOC + ~50 in <code>emit-evidence</code>	CRAP complexity-coverage math + JSON marshaling with deterministic key ordering	Python’s <code>json</code> module is the most ergonomic way to produce canonical JSON with proper escaping; <code>awk/jq</code> substitutes exist but increase the polyglot surface for negligible benefit

Layer	Technology	Version	Purpose	Why This
Policy substrate	<code>tests/TESTING.md</code> in the <i>consumer</i> repo	repo-specific	Carries coverage / mutation thresholds and arch-rule references	“Policy-driven, never hardcoded” per <code>CLAUDE.md:14</code> — <code>escape-scan.sh:78-89</code> reads <code>coverage.line</code> , <code>coverage.branch</code> , <code>mutation.kill_rate</code> from there at runtime; no hardcoded floor in script source
Pin substrate	<code>.harness-hash</code> in consumer repo	SHA-256 manifest	Tamper-evidence on engineer-owned config files	The whole point of the harness — <code>harness-hash.sh:40-60</code> pins <code>.feature</code> , dep-cruiser configs, ArchUnit tests, Stryker configs, <code>.c8rc.json</code> . Any byte-level change without a fresh <code>--init</code> triggers REFUSE
Convergence envelope	in-toto Statement v1 + <code>predicateType</code> <code>https://evals.intentsolutions.io/gate-results/v1</code>	per intent-eval-lab/specs/evidence-bundles/6.1.0/gate-results/v1 draft	Schema for Evidence Bundle rows	Reuses an open standard (in-toto attestation framework, sigstore precedent); the predicate URI is <i>frozen</i> once Rekor sees it per <code>SEMVER.md:69-77</code>
Optional signing layer	cosign attest-blob + Rekor	sigstore-current	Detached transparency-log proof	DSSE-wrap a gate-result Statement for downstream verification; gated by operator opt-in to keep zero-deps default

Layer	Technology	Version	Purpose	Why This
Optional telemetry	OTel-shaped JSON line on stderr	agent.rollout.gate-evaluated event	Re-evaluated event emission when AUDIT_HARNESS_OTEL=1	Per scripts/emit-evidence.sh:182 emits a structured signal any collector can scrape via stderr capture, but does not import an OTel SDK (keeps zero runtime deps)

System Diagram





The Critical Path

The most operationally important request is: *engineer commits a diff; pre-commit hook decides PASS / REFUSE / CHALLENGE*. This is what every consumer repo depends on. End-to-end:

1. Engineer runs `git commit -m "..."`. Husky fires `.husky/pre-commit`, which contains `pnpm exec audit-harness escape-scan --staged`.
2. `pnpm` resolves `audit-harness` to `node_modules/@intentsolutions/audit-harness/bin/audit-harness`.
3. Node parses `argv`. The dispatcher at `bin/audit-harness.js:64-77` looks up the `COMMANDS` table, finds `'escape-scan' -> escape-scan.sh` with default args `[]`.
4. `bin/audit-harness.js:90-94` chooses `bash` as the interpreter (since the script does not end in `.py`), assembles `[scriptPath, ...defaultArgs, ...userArgs]`, and spawns with `stdio: 'inherit'`.
5. `scripts/escape-scan.sh:39-46` peels `--json` off the arg list (if present), leaving primary args unchanged.
6. `scripts/escape-scan.sh:53-61` resolves the diff source: `--staged` runs `git diff --cached` into a tempfile; `--range A..B` runs `git diff A..B`; bare path is treated as a patch file; `-` reads `stdin`.
7. `scripts/escape-scan.sh:78-89` reads `tests/TESTING.md` and overrides the hardcoded fallback floors (80/70/70) if `coverage.line:`, `coverage.branch:`, `mutation.kill_rate:` keys are present.
8. `scripts/escape-scan.sh:114-145` runs three categories of pattern checks against the *added* lines (`grep '^\[~+]'`): `coverage-threshold` edits below floor, `architecture` rule bypasses (`depcruise-disable`, `@ArchIgnore`, `skip_violations`, `ignore_imports=`, `severity: "warn"`), wholesale test deletion, `.feature` file mutation.
9. `scripts/escape-scan.sh:147-155` calls the hash subcheck: `bash harness-hash.sh --verify` (silent). Exit non-zero -> `HARNESSTAMPERED REFUSE`. Also: if the diff *itself* touches any `*.feature`, that's a `REFUSE`.
10. `scripts/escape-scan.sh:157-177` runs softer pattern matches: test skip markers (`CHALLENGE`), mutation bypass markers (`CHALLENGE`), trivially-true assertions (`CHALLENGE`), smoke-only assertions (`FLAG`).

11. `scripts/escape-scan.sh:179-216` summarizes: if `JSON_OUT=1` emit a single JSON object on stdout with the gate-result shape (`gate_id`, `result`, `input_hash`, `policy_hash`, `metadata`). Always emit the human-readable `escape-scan: REFUSE=N CHALLENGE=N FLAG=N` line on stderr (or stdout in non-JSON mode).
12. Exit codes: `REFUSE > 0` -> exit 2 (pipeline halt). `CHALLENGE > 0` -> exit 1 (engineer-approved comment required). Otherwise exit 0.
13. The Node dispatcher (`bin/audit-harness.js:95-100`) forwards the child's exit code unchanged via `process.exit(code ?? 0)`. Husky sees non-zero -> abort the commit.

Failure points along this path: step 6 fails if `git` is not in `PATH` (rare in CI, possible in containerized pre-commit hooks). Step 7 fails *silently* if `tests/TESTING.md` exists but uses unexpected formatting — `escape-scan.sh:82-89` uses `grep -E '^s*coverage\.line\s*:'` and falls back to the hardcoded 80. Step 9's hash subcheck silently passes if `.harness-hash` does not exist (`harness-hash.sh:95-101` exits 3 on no manifest, which `escape-scan.sh:147` treats as non-zero -> `REFUSE` *only* if `VERIFY_HASH=1`; the wrapper is forgiving). Step 11's JSON emission uses raw `printf` with `%d` and `%s` interpolation (`escape-scan.sh:197-203`) — values like the `coverage_line_floor` integer are bash-typed and safe, but if `AUDIT_HARNESS_SIDE` ever carries a quote character (it shouldn't, but it's an env var), the JSON would corrupt.

Dependency Graph

`bin/audit-harness.js`

```
-> scripts/harness-hash.sh    (verify | init | list)
-> scripts/escape-scan.sh     (escape-scan)
-> scripts/arch-check.sh      (arch)
-> scripts/bias-count.sh      (bias)
-> scripts/gherkin-lint.sh    (gherkin-lint)
-> scripts/crap-score.py      (crap)
-> scripts/emit-evidence.sh   (emit-evidence)
```

`scripts/escape-scan.sh`

```
-> scripts/harness-hash.sh    (in-band sub-call for HARNESS_TAMPERED detection at line 148)
-> tests/TESTING.md           (reads policy thresholds)
-> .harness-hash               (reads pinned hashes)
```

`scripts/arch-check.sh`

```
-> npx dependency-cruiser      (consumer-installed, JS/TS path)
-> lint-imports                (consumer-installed, Python path)
-> vendor/bin/deptry           (consumer-installed, PHP path)
-> arch-go                     (consumer-installed, Go path)
-> ./gradlew test              (consumer-installed, JVM path)
```

`scripts/crap-score.py`

```
-> radon, coverage             (Python path)
-> gocyclo, go tool cover      (Go path)
-> cr/complextiy-report, c8    (JS/TS path)
-> rust-code-analysis-cli      (Rust path; coverage integration pending)
```

`scripts/emit-evidence.sh`

```

-> python3                (in-band JSON marshaling, lines 123-169)
-> cosign                  (optional, --sign path)
-> sigstore Rekor          (optional, --rekor-url path)

```

Build order: there is no build step. The dispatcher is plain JS that Node executes directly. Scripts are executable as-is. The Rust polyglot wrapper (`rust/`) does have a build step (`cargo build`) that `include_bytes!`'s the scripts into the binary at compile time (`rust/src/main.rs:24-32`) — this is the only place where script changes need a re-build before they propagate. Python wrapper (`python/`) bundles scripts as package data via `hatchling` and resolves them at runtime via `importlib.resources` (`python/src/intent_audit_harness/cli.py:1-32`).

What happens when each dependency is unavailable:

- **No Node:** dispatcher unusable; engineers must invoke `bash scripts/escape-scan.sh` directly or use the vendored `scripts/audit-harness` shell wrapper (`install.sh:89-147`).
- **No bash:** catastrophic — six of seven scripts fail. Mitigation: Windows users must run via WSL or Git Bash; macOS bash 3.2 may behave subtly differently from Linux bash 5 (no concrete bugs reported, but `set -euo pipefail + mapfile` patterns assume bash 4+).
- **No python3:** `crap` and `emit-evidence` fail. The other six gates work.
- **No git:** `escape-scan --staged` and `--range` fail; bare-path and stdin modes still work.
- **No cosign:** `emit-evidence --sign` exits 2 with a clear message (`scripts/emit-evidence.sh:209-212`); unsigned emission works.
- **No tests/TESTING.md:** `escape-scan` falls back to hardcoded floors (80/70/70 per `escape-scan.sh:78-80`) — this silently weakens the gate if a stricter policy was intended.
- **No .harness-hash:** `verify` exits 3 (treated as “no manifest, not an error” by adopters per `SEMVER.md:40`); `escape-scan`'s in-band hash check is skipped if `VERIFY_HASH=1` and the manifest is absent.

4. Design Decisions & Tradeoffs

Decision Log

Node dispatcher + polyglot scripts (vs. pure-language port)

- **Chosen:** 105-line Node dispatcher (`bin/audit-harness.js`) that spawns `bash` and `python3` child processes. The “real” logic lives in `scripts/*.sh` and `scripts/*.py`.
- **Over:** full TypeScript port (every gate re-implemented in TS); full Rust port; full Python port; bash-only with no Node entry point.
- **Because:** Node is the broadest “already installed in every adopter’s CI image” runtime in 2026 OSS. Scripts are language-portable (bash + python3) and battle-tested. Porting battle-tested gate logic to TypeScript would introduce *new* bugs without changing user-visible behavior, and would force consumer repos using Python or Go to install Node anyway. The chosen split lets Python consumers install via `pip install intent-audit-harness` (PyPI) and Rust consumers install via `cargo install intent-audit-harness` (crates.io) — both wrap the *same* scripts. See `CLAUDE.md:11`, `CONTRIBUTING.md:14`, `CHANGELOG.md:93`.
- **Cost:** three CLI implementations to maintain (Node, Python, Rust) and three publish workflows. Polyglot debugging (bash + python in the same script via heredoc, e.g., `emit-evidence.sh:123-169`) raises the cognitive load for new contributors. Cross-platform

Windows support is a known weak point — bash dependency makes WSL or Git Bash effectively mandatory.

- **Revisit when:** (a) the script surface stabilizes and gets large enough that a single-language re-implementation reduces maintenance net-of-rewrite cost; (b) a concrete cross-platform Windows bug forces the move per `CLAUDE.md:11`; (c) a consumer asks for an embeddable library (importable, not subprocess) — current spawn-only design makes that impossible.

Deterministic regex grammar (vs. LLM-judge classification)

- **Chosen:** REFUSE / CHALLENGE / FLAG severity grammar implemented as `grep -E` regex against added diff lines (`escape-scan.sh:105-177`).
- **Over:** an LLM-based classifier that reads a diff and answers “is this an escape attempt?”; an AST-aware rule engine (`semgrep`, `codeql`); a hand-rolled diff parser with line-pair awareness.
- **Because:** AI-generated-code detectors generalize poorly — the SOTA classifier (AST + static metrics + fine-tuned LLM) reaches F1 82.55 (`000-docs/002-RR-LAND-...` cites Suh et al. 2025). A binary detector is a *worse* posture than a three-level grammar that surfaces ambiguity to a human. Deterministic regex means the harness can never hallucinate a REFUSE on a clean diff and never miss a documented pattern. The regex set is *auditable* — adopters can read `escape-scan.sh:105-177` and know exactly what trips a refusal.
- **Cost:** false negatives on novel escape patterns the regex set has not seen. False positives on legitimate threshold *increases* that pattern-match an unrelated `[0-9]` adjacent to a coverage keyword (mitigated by floor-aware comparison at `escape-scan.sh:108-113`). No semantic understanding — a contributor who renames `tests/checkout.feature` to `tests/checkout-renamed.feature` triggers REFUSE (file deletion + no compensating addition under the same name) even though no test was deleted in spirit.
- **Revisit when:** (a) consumers report repeated false-positive churn that drives them to disable the gate; (b) an escape pattern is observed in the wild that regex cannot capture (e.g., obfuscated coverage threshold via env-var expansion); (c) the rate of new patterns added per quarter exceeds 1-2 — at that velocity an AST tool is cheaper to maintain.

Hash-pinning (vs. trust-the-PR-reviewer)

- **Chosen:** `.harness-hash` manifest with SHA-256 of every engineer-owned policy file. Any byte change without a fresh `audit-harness init` is treated as `HARNESS_TAMPERED` (exit 2) by `escape-scan` (`escape-scan.sh:147-155`) and by `verify` (`harness-hash.sh:127-136`).
- **Over:** relying on PR review to catch policy-file changes; CODEOWNERS approval gates on test-config paths; signed commits; signed policy files with detached signatures.
- **Because:** PR review is human and high-variance. CODEOWNERS gates rely on the reviewer noticing the implication of a 1-line `coverageThreshold: 70` change. Hash-pinning makes the *re-init* step the explicit moment of consent — engineer must touch two files (the policy + the manifest) in the same commit, and the manifest update is visible in the diff. This is the entire design rationale per `README.md:88-96`.
- **Cost:** every legitimate policy change becomes a two-step rebuild. Engineers learn the dance (“oh, I forgot to `pnpm exec audit-harness init`”) and occasionally bypass it (“just disable the hook this once”). The manifest itself is a SHA-256-of-file flat list — does not survive file renames (a rename produces both deletion+add, both flagged). Globbing patterns in `harness-hash.sh:40-60` are fixed (`features/**/*feature`,

.dependency-cruiser.{js,cjs}, etc.) — new policy files added by a consumer repo are *not* automatically pinned, which is a sharp edge documented at Section 8.4 below.

- **Revisit when:** (a) the rate of engineer bypass via `--no-verify` becomes detectable in CI logs (no telemetry today, so this is currently unobservable); (b) the manifest format starts to feel restrictive for repos with non-standard policy paths.

In-toto Statement v1 + frozen predicate URI (vs. proprietary envelope)

- **Chosen:** `emit-evidence` wraps every gate-result JSON in an in-toto Statement v1 with `predicateType: https://evals.intentsolutions.io/gate-result/v1` (`scripts/emit-evidence.sh:58-60`, `SEMVER.md:69-77`).
- **Over:** a bespoke Intent-Solutions-only envelope schema; SLSA provenance v1.0 (`predicateType: https://slsa.dev/provenance/v1`); SCAI predicate (Melara 2022, registered in-toto predicate); plain JSON-with-version-field.
- **Because:** in-toto Statement v1 is the open standard for attestation envelopes — sigstore tooling natively understands it, Fulcio + Rekor are first-class consumers, and the predicate URI is the namespace for the “what does this attestation say?” question. Choosing in-toto means downstream consumers (j-rig, intent-rollout-gate) can use off-the-shelf cosign/sigstore verifiers instead of bespoke Intent Solutions parsers. The custom predicate URI under `evals.intentsolutions.io` keeps Intent Solutions sovereignty over the schema body while still ridig the open envelope shell.
- **Cost:** the predicate URI is *frozen* once Rekor sees a signed Statement against it (`SEMVER.md:73-77`). Any breaking change to the predicate body shape requires minting `gate-result/v2` and running both URIs in parallel. The DNS namespace `evals.intentsolutions.io` becomes a load-bearing identity surface — CISO binding from ISEDC Session 1 (`scripts/emit-evidence.sh:39-43`) blocks the first Rekor push until DNSSEC + CAA records are pinned on that namespace. Operator discipline is required; the script does not enforce the precondition.
- **Revisit when:** (a) the in-toto spec ships v2 in a way that makes Statement v1 attestations less interoperable; (b) a regulatory regime (FIPS, EU CRA) requires a specific provenance shape that in-toto cannot carry as a predicate body; (c) Intent Solutions takes a position on standards-body filing (becomes a registered in-toto predicate type, per the SCAI precedent).

Stream split (`stdout = JSON`, `stderr = human only when --json`)

- **Chosen:** `--json` flag on every gate (v0.3.0+) routes JSON to stdout, leaves the existing human-readable summary on stderr. Without `--json`, the existing behavior is byte-identical to v0.2.0 (`SEMVER.md:60-65`).
- **Over:** a separate `--format=json` flag that replaced text output entirely; a new `audit-harness <gate> --emit-evidence` mode; a sidecar reporter daemon that consumed scripts’ stdout.
- **Because:** backward-compat is sacred for a package shipped to N adopter repos. Moving existing text output between streams is a major-version break per `SEMVER.md:25`. Splitting JSON to stdout and human-readable to stderr lets adopters parse machine output while preserving the log line their existing tail/grep CI scripts already match.
- **Cost:** stream-split is non-obvious — adopters who pipe `gate 2>&1` get JSON + human-readable interleaved and parsing breaks. The pattern of `exec 3>&1; exec 1>&2` to save+swap stdout (used in `bias-count.sh:42-44` and `gherkin-lint.sh:42-45`) is bash-

fluent but unfamiliar to engineers debugging CI failures. CI gets a third gating mode that must be tested every PR (`tests/regression/run-regression.sh:96-118`).

- **Revisit when:** (a) adopters report >2 failures from misinterpreting stream split; (b) the schema-on-stdout pattern needs to coexist with another structured output (e.g., NDJSON event stream); (c) the regression suite catches a stream-leak regression — that’s the trigger to add a stricter linter.

MIT -> Apache 2.0 relicense (v1.0.0)

- **Chosen:** relicense to Apache 2.0 in v1.0.0 (commit `fab1c42`), explicit alignment with the rest of the Intent Eval Platform ecosystem (`intent-eval-lab`, `intent-eval-core`). Existing 0.x npm tarballs remain MIT (immutable per npm policy).
- **Over:** stay on MIT; dual-license MIT + Apache; switch to MPL or LGPL.
- **Because:** Apache 2.0’s explicit patent grant (clause 3) is meaningful for a package that asserts opinions about test-quality enforcement — patents around AI-test-tampering detection are not impossible. Aligning every IEP repo on one OSI license simplifies the convergence story for partners (one license to review, not three). MIT lacks explicit patent grant.
- **Cost:** a MAJOR version bump cut purely for legal clarity — no code, behavior, CLI, or runtime-dep changes (per `CHANGELOG.md:11-22`). Some consumers may have license scanners that auto-block “MAJOR version bumps” pending review, slowing uptake. NOTICE file must now ship in the tarball per Apache 2.0 §4 — v1.0.0 missed this and v1.0.1 was a packaging-only patch fix.
- **Revisit when:** never (relicense is sticky); the cost of *another* relicense is non-trivial because every existing version of the package would still be under the prior license.

Triple-publish (npm + PyPI + crates.io)

- **Chosen:** ship the same CLI surface under three package names — `@intentsolutions/audit-harness` (npm), `intent-audit-harness` (PyPI, `python/`), `intent-audit-harness` (crates.io, `rust/`). Plus `install.sh` for any other language.
- **Over:** npm-only (force Python repos to install Node); npm + curl-install only (skip PyPI/crates); language-specific re-implementations.
- **Because:** enforcement-travels-with-the-code (umbrella `CLAUDE.md`, Testing SOP) means the harness must be a dev-dependency in the *target* ecosystem’s manifest. A Python team will pin via `pyproject.toml`, not `package.json`. Triple-publish lets each ecosystem use its idiomatic install path. Rust binary statically embeds scripts via `include_bytes!` (`rust/src/main.rs:24-32`) producing a self-contained tool; Python uses `importlib.resources` (`python/src/intent_audit_harness/cli.py`).
- **Cost:** three publish workflows, three version histories, three changelogs to keep in sync — and *they have drifted*. As of 2026-05-20: `package.json` is 1.0.1, `version.txt` is 0.2.0, `python/pyproject.toml` is 0.1.0, `rust/Cargo.toml` is 0.1.0. Calling `audit-harness --version` returns different strings depending on which package the consumer installed. Each wrapper’s PyPI/crates listing shows it as still-MIT-licensed even though the canonical license is Apache 2.0 (per `python/pyproject.toml:8` and `rust/Cargo.toml:9`).
- **Revisit when:** (a) the version drift causes a support ticket — sooner rather than later; (b) the `install.sh` vendoring pattern proves more popular than the package-manager paths (it’s currently the long-tail catch-all per `README.md:46-50`); (c) a fourth ecosystem (Go, Ruby) generates enough adoption signal to justify another wrapper.

What Was Deliberately Not Built

- **No mutation-testing dispatcher.** The harness pins `stryker.conf.json` and `stryker.config.js` in `.harness-hash` (`harness-hash.sh:58-59`) but does not *run* Stryker / PIT / mutmut / cargo-mutants. The kill-rate floor is a policy threshold in `tests/TESTING.md`, not a gate computed by the harness. Rationale: mutation testing is performance-expensive (Sánchez et al. 2024 — see `000-docs/002-RR-LAND-...` — find performance is the #1 barrier); running it in pre-commit is unworkable; running in CI is consumer responsibility, the harness only enforces the threshold the consumer's CI reports.
- **No SAST / SBOM / secret-scan layer.** L2 of the 7-layer taxonomy is explicitly out of scope. Semgrep, Trivy, Gitleaks, etc. are consumer-installed; the harness does not orchestrate them. Rationale: L2 tools have their own ecosystems and own gate semantics; bolting them onto the L1+L3 surface dilutes the harness's identity.
- **No LLM-in-the-loop gate.** Every gate is regex / AST-pinned / hash-pinned / arithmetic — there is no place in the codebase where Claude or any other model decides PASS/FAIL. Rationale: per `CLAUDE.md:11` “Deterministic shell/python is sufficient (no LLM-in-the-loop).” Determinism is the load-bearing property.
- **No telemetry collection.** `AUDIT_HARNESS_OTEL=1` emits an OTel-shaped JSON line on `stderr`, but the script does not import an OTel SDK and does not push to a collector (`scripts/emit-evidence.sh:182-187`). Operator must scrape `stderr`. Rationale: zero run-time deps is a hard rule (`CLAUDE.md:12`).
- **No web UI / dashboard.** The harness emits JSON; consumers route it. No bundled visualization.
- **No automatic version pinning of consumer arch-checkers.** `arch-check.sh` calls whatever `dependency-cruiser` is in the consumer's `node_modules` — does not pin the checker version itself. Rationale: pin discipline is consumer-side via `pnpm-lock.yaml`, harness should not get in that business.
- **No native Windows support.** Bash is mandatory (`set -euo pipefail, [[ ]]`, process substitution); Windows users go through WSL or Git Bash. Rationale: per `CLAUDE.md:11` “Don't port to TypeScript unless there's a concrete reason (cross-platform Windows bug, etc.)” — no concrete Windows bug has been reported.

Assumptions the Architecture Rests On

- **Bash 4+ in the consumer environment.** `mapfile -t, [[ -v ]]`, `\${var,,}` lowercasing, and `set -euo pipefail` behavior assume bash ≥ 4 . macOS users on default `/bin/bash` 3.2 will hit subtle bugs; the harness assumes `brew install bash` or use of Git Bash 5.x on macOS. Not documented.
- **Python 3.10+ in the consumer environment** when using `crap` or `emit-evidence --sign`. `crap-score.py:39` uses `str | None` (PEP 604 union syntax) which is Python 3.10+; `python/pyproject.toml:8` declares `requires-python = ">=3.8"` — that's a documentation drift; `CONTRIBUTING.md:12` says 3.10+. CI tests 3.12.
- **git in PATH.** `escape-scan --staged` and `--range` rely on git; missing git silently breaks the diff acquisition.
- **tests/TESTING.md with the expected key format** (`coverage.line: 80,mutation.kill_rate: 70`). Mis-formatted files fall back to the hardcoded floors silently — `escape-scan.sh:82-89` uses lenient `grep -E` matching and any parse failure leaves the default 80/70/70 in place. A consumer who *thinks* they've lowered their floor to 60 may still be enforcing 80, or vice versa.

- **No symlink shenanigans in policy paths.** `harness-hash.sh` uses `sha256sum` on raw paths — a symlink pointing at a malicious file would hash the target, not the link. The `SECURITY.md:64` threat model flags “symlink traversal + path-escape are real concerns” but the script doesn’t dereference-check.
- **Cosign in PATH when `--sign` is used.** `emit-evidence.sh:209-212` checks command `-v cosign` and exits 2 with a clear message if missing.
- **`evals.intentsolutions.io` namespace will remain under Intent Solutions control forever.** The predicate URI is frozen once any signed Statement is pushed to Rekor — losing DNS control of that namespace would compromise every historical attestation.

5. Directory Structure

Layout

```
audit-harness/
+-- AGENTS.md                # bd workflow for AI agents working in this repo
+-- CHANGELOG.md             # Keep-a-Changelog format; release notes per tag
+-- CLAUDE.md                # Claude Code session guidance; design rules; release fl
+-- CODE_OF_CONDUCT.md      # standard CoC
+-- CONTRIBUTING.md          # local dev setup + PR conventions
+-- LICENSE                  # Apache 2.0 since v1.0.0
+-- NOTICE                  # Apache 2.0 §4 attribution; added v1.0.1
+-- README.md                # the "what it is, install, quick usage" entry point
+-- SECURITY.md               # vuln-report email + threat model + severity table
+-- SEMVER.md                # SemVer commitment doc (the consumer-promise contract)
+-- SUPPORT.md               # how to get help; response time table
+-- install.sh               # curl-based vendoring installer for non-Node repos
+-- package.json              # npm manifest; "version": "1.0.1"; engines: node>=18
+-- version.txt               # plain-text version; says "0.2.0" - out-of-sync with pa
+-- .github/
|   +-- CODEOWNERS            # @jeremylongshore on /scripts, /bin, /.github/workflows
|   +-- dependabot.yml        # weekly github-actions + npm bumps
|   +-- FUNDING.yml           # GH sponsors link
|   +-- ISSUE_TEMPLATE/
|       |   +-- bug_report.md  # require repro + version up front
|       |   +-- config.yml     # disables blank issues; routes to email for security
|       |   +-- feature_request.md # require "why deterministic shell/python is sufficient"
|   +-- PULL_REQUEST_TEMPLATE.md # explicit Consumer-Repo-Impact section
|   +-- workflows/
|       +-- ci.yml             # the only workflow: self-check + shellcheck + python-sy
+-- 000-docs/                  # per Doc Filing Standard v4.3
|   +-- 001-DR-DESIGN-evidence-bundle-envelope-design-notes.md # Phase A design notes for AH-
|   +-- 002-RR-LAND-upgrade-landscape.md                       # competitor + academic landsc
|   +-- 003-AA-AUDT-appaudit-devops-playbook.md                # this document
+-- bin/
|   +-- audit-harness.js      # 105-LOC Node dispatcher; the user-facing entry point
+-- scripts/                  # the actual product surface
```



```

|   +-- arch-check.sh                # 167 LOC; dispatches dep-cruiser / import-linter / depts
|   +-- bias-count.sh                # 136 LOC; advisory grep-based test-bias counter
|   +-- crap-score.py                # 426 LOC; multi-language CRAP scorer (radon/gocyclo/c8/
|   +-- emit-evidence.sh             # 256 LOC; in-toto Statement v1 wrapper + cosign sign + l
|   +-- escape-scan.sh               # 216 LOC; the main AI-containment gate
|   +-- gherkin-lint.sh              # 143 LOC; awk-fallback Gherkin lint
|   +-- harness-hash.sh              # 156 LOC; SHA-256 manifest of pinned policy files
+-- tests/
|   +-- fixtures/
|       |   +-- clean.diff           # diff that should PASS escape-scan
|       |   +-- empty.diff
|       |   +-- gate-result.schema.json # local copy of intent-eval-lab's gate-result schema
|       |   +-- refuse.diff          # diff with --cov-fail-under=10 (below floor 80) -> REFUS
|   +-- regression/
|       +-- run-regression.sh         # 203 LOC; 11-check backward-compat suite for --json + er
+-- python/
|   +-- pyproject.toml               # PyPI wrapper; same scripts re-bundled as package data
|   +-- PUBLISH.md
|   +-- README.md
|   +-- src/intent_audit_harness/
|       +-- __init__.py              # __version__ = "0.1.0"
|       +-- __main__.py
|       +-- cli.py                   # Python dispatcher mirroring the Node CLI
|       +-- scripts/                 # package-data copy of canonical scripts/
+-- rust/                             # crates.io wrapper; scripts embedded at compile time
|   +-- Cargo.lock
|   +-- Cargo.toml                   # version "0.1.0" - out-of-sync
|   +-- LICENSE                       # still MIT per Cargo.toml license field
|   +-- PUBLISH.md
|   +-- README.md
|   +-- scripts/                     # symlinked or copied; include_bytes! targets
|   +-- src/main.rs                  # 156 LOC; cache-extracts embedded scripts to $XDG_CACHE
|   +-- target/                       # gitignored build output

```

Load-Bearing Files

These are the files that, if subtly corrupted, break the security or correctness of every consumer repo. Treat them as airline-grade: PR review them deeply, run shellcheck, run the regression suite locally, and don't merge after midnight.

1. **scripts/escape-scan.sh** (216 LOC) — the AI-containment heart. Bug in any of the REFUSE-pattern regexes silently lets a threshold-lowering diff through. Bug in the floor parser (lines 82–89) silently uses hardcoded defaults. The CI synthetic-input self-check (`.github/workflows/ci.yml:53–78`) is the canary; if that step ever passes when it shouldn't, the harness is *worse than nothing* because adopters trust the gate.
2. **scripts/harness-hash.sh** (156 LOC) — the tamper-detection substrate. The PATTERNS array (lines 40–60) is the closed list of pinned-file globs. Adding a new policy artifact category (e.g., a custom rule config) requires editing this array — there is no consumer-side

extension mechanism. The `--verify` output format is hashed by `escape-scan.sh:147-150` to detect tampering, so the format itself is load-bearing.

3. **scripts/emit-evidence.sh** (256 LOC) — the convergence-layer envelope emitter. The Python heredoc at lines 123-169 encodes the in-toto Statement v1 contract. Any change to the subject/predicate shape silently breaks `intent-eval-lab` schema validation downstream. The predicate URI `https://evals.intentsolutions.io/gate-result/v1` at line 55 is frozen post-Rekor-push — accidentally renaming or path-shifting it is unrecoverable.
4. **bin/audit-harness.js** (105 LOC) — the dispatcher. The `COMMANDS` table at lines 15-25 is the canonical mapping; adding a new gate requires adding a row here AND in `python/src/intent_audit_harness/cli.py:COMMANDS` AND `rust/src/main.rs:resolve_command()`. Drift between these three is a real failure mode.
5. **SEMVER.md** — the consumer contract. The “what we will never do” section is the durable promise — violations break adopter CI without warning. If a PR changes any behavior listed in the “stable contracts” tables, this file *must* be updated in the same commit.
6. **tests/regression/run-regression.sh** — the backward-compatible ratchet. Section 1 (text-mode parity) is the only guard against accidental output drift on the existing v0.2.0 surface. If this script is weakened (e.g., loosening an exact-string check to a regex), the SemVer “no silent output drift” promise becomes unenforceable.
7. **.github/workflows/ci.yml** — the gating pipeline. Note: the `audit-harness verify`, `audit-harness list`, and `harness-hash --verify` self-check steps currently `|| true` to tolerate exit-3 (“no manifest”). That’s a known weakness — the repo does not yet self-pin.
8. **package.json** — the npm-side source of truth for the version string surfaced via `audit-harness --version`. Mismatches with `version.txt`, `python/pyproject.toml`, and `rust/Cargo.toml` are unresolved drift.
9. **install.sh** (175 LOC) — the curl-pipe-bash installer. Anyone running `curl ... | bash` is trusting this script implicitly; a malicious modification would compromise every non-Node consumer at next install. Pinned by HTTPS-only fetch and a CHECKSUM-less GitHub-release-tarball pattern — there is *no* signature verification on the tarball (Section 8.2).
10. **scripts/crap-score.py** (426 LOC) — the only multi-language scorer. The dispatch table at lines 281-286 is the authoritative list of supported languages; a bug in `score_python` or `score_go` silently produces zero-coverage scores for that language.

6. Getting Started

Prerequisites

Table 4.

Tool	Version	Install	Verify
Node.js	>= 18	<code>nvm install 22</code> (per .github/workflows/ci.yml#L22 matrix)	<code>node --version -> 22.x</code>
Bash	4+ (5 recommended)	macOS: <code>brew install bash</code> ; Linux: pre-installed	<code>bash --version -> GNU bash, version 5.x</code>
Python 3	3.10+ (CI uses 3.12)	<code>pyenv install 3.12</code> or system pkg manager	<code>python3 --version -> Python 3.12.x</code>
Git	any recent	system pkg manager	<code>git --version</code>
pnpm (optional, for <code>pnpm exec</code> consumers)	8+	<code>npm i -g pnpm</code>	<code>pnpm --version</code>
shellcheck (optional, for contributors)	latest	<code>apt install shellcheck</code> / <code>brew install shellcheck</code>	<code>shellcheck --version</code>
cosign (optional, for <code>emit-evidence --sign</code>)	latest	https://docs.sigstore.dev/cosign/installation/	<code>cosign --version</code>

Zero to Running

This is the path for someone who has just cloned the repo and wants to verify it works locally. Total time: 3-5 minutes.

1. `git clone https://github.com/jeremylongshore/audit-harness.git && cd audit-harness` — expect a working tree with `bin/`, `scripts/`, `tests/`, `python/`, `rust/`, and 12 root-level markdown files.
2. `npm install` — expect “added 0 packages” (the package has zero runtime dependencies; dev-deps may show as 0 too since none are declared in `package.json:39-49`). If anything was installed, that’s drift — investigate.
3. `node bin/audit-harness.js --version` — expect 1.0.1 (matches `package.json`).
4. `node bin/audit-harness.js --help` — expect the usage banner from `bin/audit-harness.js:28-62`.
5. `node bin/audit-harness.js list` — expect `harness-hash: no manifest` (run `--init`) and exit code 3. This is *correct*: this repo does not pin itself yet (Section 11 finding).
6. `bash scripts/escape-scan.sh --no-hash tests/fixtures/clean.diff` — expect `escape-scan: REFUSE=0 CHALLENGE=0 FLAG=0` and exit 0.
7. `bash scripts/escape-scan.sh --no-hash tests/fixtures/refuse.diff` — expect `[REFUSE] coverage fail_under lowered below policy floor (80) ...` on `stderr`, `escape-scan: REFUSE=1 ... summary`, and exit 2.

8. `bash tests/regression/run-regression.sh` — expect `PASS=11 FAIL=0` if `jsonschema` is installed (`pip install jsonschema`), or `PASS=8 FAIL=0` with a “Section 3 skipped” warning otherwise.

If steps 6-7 produce the wrong exit codes, the gate logic has regressed — do not proceed.

Running a gate on a real repo

This is the path for someone trying to *use* the harness in another repo.

1. From the target repo’s root: `pnpm add -D @intentsolutions/audit-harness` (Node) or `pip install intent-audit-harness` (Python) or `cargo install intent-audit-harness` (Rust) or `curl -sSL https://raw.githubusercontent.com/jeremylongshore/audit-harness/main/install | bash` (everything else).
2. Create `tests/TESTING.md` with at least:


```
coverage.line: 80
coverage.branch: 70
mutation.kill_rate: 70
```
3. `pnpm exec audit-harness init` (or `scripts/audit-harness init` for vendored) — this writes `.harness-hash` pinning every `*.feature`, `.dependency-cruiser.cjs`, `.importlinter`, `stryker.conf.json`, etc. that exists in the repo.
4. `pnpm exec audit-harness list` — confirm the pinned files match expectations.
5. Wire pre-commit:


```
# .husky/pre-commit
pnpm exec audit-harness escape-scan --staged
pnpm exec audit-harness verify
```
6. Wire CI (see `README.md:64-75` for canonical example).
7. Test the gate: stage a `coverage.line: 60` change to `tests/TESTING.md` and try to commit. Expect REFUSE.

Common Setup Problems

Table 5.

Symptom	Cause	Fix
<code>audit-harness: command not found</code> after install	<code>node_modules/.bin</code> not on <code>PATH</code> (running outside <code>pnpm exec</code>)	Use <code>pnpm exec audit-harness ...</code> or <code>npx audit-harness ...</code>
<code>bash: set: -o: invalid option</code> on macOS	macOS default <code>/bin/bash</code> is 3.2; <code>set -euo pipefail</code> is fine but <code>mapfile</code> and other 4+ idioms fail	<code>brew install bash</code> ; ensure which <code>bash</code> resolves to homebrew <code>bash</code>

Symptom	Cause	Fix
<code>escape-scan</code> : REFUSE=0 on a diff that <i>should</i> refuse	Floor parser silent-fallback (<code>escape-scan.sh:82-89</code>) — <code>tests/TESTING.md</code> format is off	Confirm <code>coverage.line: 80</code> exact format; whitespace-after-colon is fine, missing colon falls back
HARNESS_TAMPERED on legitimate policy change	Engineer forgot <code>audit-harness init</code> after editing <code>tests/TESTING.md</code> or other pinned file	<code>pnpm exec audit-harness init</code> + <code>git add .harness-hash</code> in same commit as the policy change
<code>cosign</code> not installed when running <code>emit-evidence --sign</code> <code>crap</code> returns no scores	<code>cosign</code> is operator-installed Language scorer (radon, gocyclo, etc.) not installed in consumer repo	https://docs.sigstore.dev/cosign Install the per-language tool from <code>README.md:130-140</code> table
<code>emit-evidence</code> rejects input with missing required keys: <code>['gate_id', ...]</code>	Piping a non-JSON or non-gate-result envelope	Ensure upstream gate was called with <code>--json</code> ; not <code>2>&1-merged</code>
Version mismatch (<code>pip</code> vs <code>npm</code>)	Three-way version drift (Section 8.6)	Treat <code>npm</code> 's 1.0.1 as canonical; PyPI/crates may lag
<code>audit-harness verify</code> exits 3 in CI	No <code>.harness-hash</code> exists in the repo	Run <code>audit-harness init</code> once locally, commit <code>.harness-hash</code> , re-run CI

7. Operations

Command Map

Table 6.

Task	Command	Notes
Run locally (smoke)	<code>node bin/audit-harness.js --version</code>	Should print 1.0.1
Run a gate locally	<code>bash scripts/<gate>.sh ...</code>	Direct script invocation bypasses the Node dispatcher
Run regression suite	<code>bash tests/regression/run-regression.sh</code>	11 checks; needs <code>pip install jsonschema</code> for Section 3
Lint scripts	<code>shellcheck scripts/*.sh</code>	CI runs this advisory (<code> true</code>)
Python syntax check	<code>find scripts -name '*.py' -print0 \ xargs -0 -n1 python -m py_compile</code>	CI runs this as a gate
Bump version (npm)	<code>npm version patch \ minor \ major</code>	Updates <code>package.json</code> only; <i>does not</i> sync <code>version.txt</code> / Python / Rust
Publish (npm)	<code>pnpm publish</code>	Needs <code>~/.npmrc</code> with org-scope token; runs <code>prepublishOnly</code> smoke first
Publish (PyPI)	per <code>python/PUBLISH.md</code> (Hatch backend)	Manual; not part of CI
Publish (crates.io)	per <code>rust/PUBLISH.md</code>	Manual; not part of CI
Tag and push	<code>git push --follow-tags</code>	After <code>npm version</code>
Test escape-scan against synthetic diff	<code>bash tests/regression/run-regression.sh</code> Section 1 + 2	Or stage a <code>--cov-fail-under=10</code> diff and run <code>bash scripts/escape-scan.sh --staged</code>
View CI status	<code>gh run list --repo jeremylongshore/audit-harness --limit 10</code>	Or GitHub Actions UI
Rollback (npm)	<code>npm deprecate @intentsolutions/audit-harness@<bad-version> '<reason>'</code>	npm tarballs are immutable; must delete; must publish a fixed version and deprecate the bad one

Deployment

This package does not deploy to a runtime environment — it deploys to package registries. The “deployment” is the publish step.

Pre-flight checklist:

- All CI green on `main` (self-check on 3 Node versions, shellcheck advisory, python-syntax gating, regression suite gating)
- `CHANGELOG.md` updated with the new version, date, and Keep-a-Changelog-formatted notes
- `SEMVER.md` updated if CLI surface changed
- `package.json` version bumped (and `version.txt`, `python/pyproject.toml`, `rust/Cargo.toml` if releasing those wrappers)
- `git status` clean; `git log` shows the bump commit at HEAD
- `node bin/audit-harness.js --version` matches `package.json` version (this is the `prepublishOnly` smoke check)

Execution (npm — the canonical path):

```
npm version patch           # or minor, or major; updates package.json + creates a tag
pnpm publish                # publishes to npm; requires ~/.npmrc with token
git push --follow-tags     # pushes the tag to GitHub
```

Verification (post-publish):

```
npm view @intentsolutions/audit-harness version    # should match the new tag
npm install -g @intentsolutions/audit-harness@latest # in a clean env
audit-harness --version                             # confirms the new version
```

Rollback protocol:

npm tarballs are *immutable*. There is no “delete a release.” If a bad version ships:

```
# 1. Immediately deprecate the bad version
```

```
npm deprecate @intentsolutions/audit-harness@<bad-version> 'critical regression - use <prev-ver>'
```

```
# 2. Cut a hot-fix patch on a branch off the last-known-good tag
```

```
git checkout -b hotfix/<bad-version>-revert <prev-version>
```

```
# ... apply fix ...
```

```
npm version patch
```

```
pnpm publish
```

```
git push --follow-tags
```

```
# 3. Announce: GitHub release notes, CHANGELOG entry, optionally a tweet from @intentsolutions
```

For PyPI: `pip install` resolves to the latest non-yanked version; use `pip admin` UI on PyPI to *yank* (not delete) the bad release. For crates.io: `cargo` prevents over-publishing the same version, but `cargo yank` flags a version as un-resolvable for new dependents.

Monitoring & Alerting

- **Dashboards:** not configured. npm provides public download counts at <https://www.npmjs.com/package/@intentsolutions/audit-harness>; PyPI provides them via <https://pypi.org/project/intent-audit-harness/#history>; crates.io likewise.

- **SLIs/SLOs:** not formally defined. Implicit: CI green on `main`; regression suite passes; no high-severity `SECURITY.md` report open.
- **On-call:** not established. Sole maintainer (Jeremy Longshore) per `CODEOWNERS`; `SUPPORT.md` sets best-effort response times (24h security, 3 business days bugs).
- **Issue triage:** GitHub Issues with the bundled templates (`/.github/ISSUE_TEMPLATE/`); bug reports must include version + Node version + OS + repro per `bug_report.md`.
- **Dependabot:** weekly Monday bumps for `github-actions` and `npm` ecosystems (`.github/dependabot.yml`); no PyPI / crates.io ecosystems wired (gap).

Incident Response

Table 7.

Severity	Definition	Response Time	Playbook
P0	Escape-scan bypass in the wild (a published version lets a known-pattern through); harness corruption (a tampered tarball is on npm); credential leak in published assets	Immediate	(1) npm deprecate the bad versions with a clear reason; (2) cut a hot-fix patch from the last-known-good tag; (3) email security@intentsolutions.io thread; (4) open a SECURITY-labeled CHANGELOG entry on the next release; (5) notify the IEP umbrella issue in intent-eval-lab
P1	Regression in a gate's exit code or output stream on existing input (SemVer break)	24 hours	(1) Reproduce locally with the failing fixture; (2) bisect from the last passing tag; (3) fix + add a regression case to tests/regression/run-regressi ; (4) hot-fix patch release; (5) update <code>SEMVER.md</code> if the contract was unclear

Severity	Definition	Response Time	Playbook
P2	New false-positive REFUSE on a legitimate diff; documentation drift; version-string mismatch between ecosystems	1 week	(1) Add a fixture demonstrating the false positive to <code>tests/fixtures/</code> ; (2) tighten the regex or fix the parser; (3) regular patch release
P3	Polish issues; new gate feature requests	Best-effort	Open as a <code>feature_request</code> issue; gated by Phase B per <code>CLAUDE.md:34-40</code>

8. Things That Will Bite You

Ordered by likelihood $\times$ impact.

8.1 Hash-pin manifest churn — engineer forgets `init` after a legitimate policy change

- **Symptom:** pre-commit hook fails with `HARNESS_TAMPERED: pinned artifact changed` for a commit the engineer believes is legitimate (e.g., they intentionally edited `tests/TESTING.md` to raise the coverage floor).
- **Cause:** `.harness-hash` records the SHA-256 of every pinned file at the last `audit-harness init`. Any byte change without a fresh `init` is treated as tampering by both `escape-scan.sh:147-155` and `harness-hash.sh --verify` (exit 2).
- **Fix:** `pnpm exec audit-harness init` to refresh the manifest, then `git add .harness-hash` and recommit in one shot with the policy change.
- **Prevention:** train engineers on the two-step pattern: “edit policy, then `init`, then commit both.” Add a `--also-init` pre-commit option in a future release (AH-? candidate; not currently scheduled).

8.2 `install.sh` tarball has no signature verification

- **Symptom:** a malicious actor MITMs the `curl-pipe-bash` install (network attacker, not a GitHub compromise) and substitutes a tampered tarball; the consumer’s `.audit-harness/` is now backdoor-pinned without any detection.
- **Cause:** `install.sh:52-67` fetches `https://github.com/jeremylongshore/audit-harness/archive/ref` via plain `curl`. There is *no* checksum, signature, or cosign verification. The script trusts HTTPS + GitHub’s host.
- **Fix:** short-term, pin to a specific known-good version (`AUDIT_HARNESS_VERSION=v1.0.1 curl ...`); medium-term, ship a SHA-256 list per release in `CHECKSUMS.txt` and have

`install.sh` verify before extraction; long-term, sign release tarballs via cosign and verify keyless during install.

- **Prevention:** do not pipe `curl | bash` over untrusted networks. Mirror the install logic into the consumer repo if regulatory posture demands provenance.

8.3 Stream split confusion — `gate 2>&1 | jq` does not work in `--json` mode

- **Symptom:** an adopter writes a CI script like `audit-harness escape-scan --staged --json 2>&1 | jq '.result'` and gets parse errors. The JSON parser sees the human-readable `escape-scan: REFUSE=N ...` line on the merged stream.
- **Cause:** `--json` deliberately splits streams: stdout is JSON, stderr is the existing human-readable summary (`SEMVER.md:60-65`). Merging streams defeats the design.
- **Fix:** redirect stderr to `/dev/null` or a log file: `audit-harness escape-scan --staged --json 2>/tmp/scan.log | jq '.result'`.
- **Prevention:** document the stream split prominently in README quick-usage (currently only in `SEMVER.md:60-65`).

8.4 PATTERNS array in `harness-hash.sh` is closed — new policy types not auto-pinned

- **Symptom:** a consumer adds a custom rule file (e.g., `.eslint-arch-rules.json`) and expects it to be pinned. It isn't. AI agents can mutate it silently.
- **Cause:** `harness-hash.sh:40-60` hardcodes the glob set. There is no consumer-side extension mechanism today.
- **Fix:** short-term, fork or PR a new pattern. Medium-term: support a `.harness-hash-patterns` file in the consumer repo that the script reads. (No bead filed; would be a v0.4 / v1.1 minor.)
- **Prevention:** if you add a custom policy artifact, search `scripts/harness-hash.sh` for an existing pattern that already covers it (e.g., `.feature` globs).

8.5 SOPS-dotenv-eval-leak pattern is NOT in this repo but adjacent

- **Symptom:** a contributor adds a CI step that sources SOPS-encrypted env vars via `eval "$(sops -d ... | sed 's/^/export /')"`. Comments and blank lines become `export # ...` or bare `export`, and bare `export` dumps every exported variable to stdout. If CI captures that stdout (notification step, cron mail), every secret leaks.
- **Cause:** documented in the umbrella `~/000-projects/CLAUDE.md` (also in user-level memory at OPS-8ft 2026-05-02). Not currently a vector in this repo because the harness has zero secrets in its own CI, but if `emit-evidence --sign --rekor-url` against keyless cosign starts pulling OIDC tokens in CI, the pattern becomes relevant.
- **Fix:** use the anchored-regex form: `eval "$(sops -d ... | sed -nE 's/^([A-Za-z_][A-Za-z0-9_]*)=(.\\1=\\2/p')"`.
- **Prevention:** if a CI step needs SOPS env, use the canonical wrapper from the umbrella `CLAUDE.md`, not an inline eval.

8.6 Three-way version drift: `package.json` vs `python/pyproject` vs `rust/Cargo` vs `version.txt`

- **Symptom:** a consumer asks “what version of audit-harness am I running?” and gets different answers from `npm`, `pip`, and `cargo`. The `version.txt` file at the repo root claims 0.2.0 while `package.json` says 1.0.1.
- **Cause:** each ecosystem has its own version source of truth; the `npm` path (`package.json`) is the one that bumps with `npm version`. Python and Rust wrappers have manual versioning

steps documented in `python/PUBLISH.md` and `rust/PUBLISH.md` and they have not been re-published since v0.1.0. `version.txt` appears stale at 0.2.0 (likely a `/release` artifact that was not removed).

- **Fix:** short-term, treat `package.json` as canonical and document the lag explicitly in README. Medium-term, wire a `scripts/sync-versions.sh` step into the `npm version` post-commit hook that updates the other three files; or move all four to read from `package.json` at install time.
- **Prevention:** do not infer license or version from anywhere other than `package.json` for npm consumers, `python/pyproject.toml` for PyPI consumers, etc.

8.7 macOS bash 3.2 vs Linux bash 5

- **Symptom:** `audit-harness escape-scan --staged` works on Linux CI but produces silent garbage on a developer's macOS laptop using `/bin/bash (3.2)`.
- **Cause:** `set -euo pipefail` is fine. `mapfile -t` is bash 4+. `[[ -v ]]` is bash 4.2+. Process substitution combined with `set -u` may behave subtly differently across versions. The scripts assume bash 4+ but do not check at startup.
- **Fix:** add a `#!/usr/bin/env bash` (already there) + an explicit version-check preamble at script top: `if (( BASH_VERSINFO[0] < 4 )); then echo "audit-harness: bash 4+ required"; exit 2; fi.`
- **Prevention:** macOS users `brew install bash` and ensure `which bash` resolves to `homebrew bash` (currently undocumented).

8.8 escape-scan --no-hash flag silently passes when run before init

- **Symptom:** a contributor wires `audit-harness escape-scan --staged` into CI without first running `init`, expecting the harness to refuse. But every diff PASSES because the in-band hash check is skipped (`escape-scan.sh:147-150` checks `[[ -f "$ROOT/.harness-hash" ]]` and silently skips otherwise).
- **Cause:** by design — `escape-scan` should be runnable before any manifest exists, so it doesn't *require* a manifest. But the silent skip means the operator doesn't know they're missing a layer of protection.
- **Fix:** short-term, the CI should also run `audit-harness verify` separately (the canonical pre-commit example in README does this). Medium-term, `escape-scan` should print a warning to `stderr` when running without a manifest.
- **Prevention:** always pair `escape-scan` with `verify` in pre-commit and CI. Both are needed.

8.9 Predicate URI is frozen forever once any signed Statement hits Rekor

- **Symptom:** after months of `emit-evidence --sign --rekor-url` runs, someone realizes the predicate body shape needs a breaking change. They cannot just change the URI body shape — every historical attestation references the old shape.
- **Cause:** per `SEMVER.md:73-77`, “We will never silently change the body shape under the same URI.” Breaking changes mint a new URI (`gate-result/v2`); both must be supported in parallel during transition.
- **Fix:** bump the URI path component, e.g., `https://evals.intentsolutions.io/gate-result/v2`; `emit-evidence` supports both; consumers gradually migrate.
- **Prevention:** review the predicate body shape against `intent-eval-lab's evidence-bundle SPEC` before the *first* signed Rekor push. The CISO binding (DNSSEC + CAA precondition) is meant to slow this down deliberately.

8.10 self-check CI tolerates exit-3 — harness does not protect itself

- **Symptom:** a malicious contributor edits `scripts/escape-scan.sh` to weaken a pattern. The PR runs through CI; the self-check step (`.github/workflows/ci.yml:36-51`) tolerates exit-3 (“no manifest”) via `|| true`; the synthetic-input check at lines 53-78 still passes because the weakened pattern wasn’t the one targeted. The bad PR merges.
- **Cause:** the repo does not yet initialize a `.harness-hash` manifest against its own scripts (per the CI comment at `.github/workflows/ci.yml:42-46`: “the long-term fix... initialize a self-pinning manifest on the repo’s own `.feature` files + script configs”).
- **Fix:** tracked in CLAUDE.md and the CI comment; create `.harness-hash` against `scripts/*.sh`, `bin/audit-harness.js`, `scripts/*.py`, and treat any in-PR change as requiring an explicit `audit-harness init` in the same commit.
- **Prevention:** until self-pinning lands, CODEOWNERS protection on `/scripts/` and `/bin/` is the only defense — strict approval gating, no merge without review.

9. Security & Access

Access Control

Table 8.

Role	Purpose	Permissions	MFA
Maintainer (@jeremylongshore)	Sole CODEOWNER per <code>github/CODEOWNERS</code>	Admin on repo; npm publish; PyPI publish; crates.io publish	Required (GitHub)
Contributors	Open issues, open PRs	Read; cannot merge without review	n/a
Dependabot bot	Weekly bumps for GitHub Actions + npm	Authored PRs; cannot merge	n/a
Security reporters	Vuln disclosure	Email <code>security@intentsolutions.io</code>	n/a

Secrets

- **Where:** no secrets in the repo. `~/.npmrc` (local to maintainer) carries the org-scope npm token for publish. PyPI / crates.io tokens similarly local. Cosign keys (if used for `emit-evidence --sign --key`) are operator-managed; cosign keyless (Fulcio OIDC) avoids long-lived keys entirely.
- **Rotation:** not documented. Recommended: rotate npm publish token every 90 days; same for PyPI / crates.io. If the maintainer’s laptop is compromised, all three publish surfaces are exposed.
- **Emergency access:** not established. Bus-factor = 1. If the sole maintainer is unavailable, no one can publish a hot-fix release until access is transferred.

Honest Security Assessment

Implemented:

- CODEOWNERS gating on `/scripts/`, `/bin/`, `/.github/workflows/`, `SECURITY.md`, `dependabot.yml` (`.github/CODEOWNERS`)
- Dependabot weekly bumps with PR limit (`.github/dependabot.yml`)
- Apache 2.0 with explicit patent grant (`LICENSE`)
- NOTICE file shipped in tarball per Apache 2.0 §4 (post-v1.0.1)
- Documented threat model and severity table (`SECURITY.md:43-50`)
- `security@intentsolutions.io` disclosure channel with 24-hour acknowledgment SLO
- Zero runtime dependencies in the Node package — minimal supply-chain surface
- Test corpus that validates the gate's REFUSE behavior against synthetic inputs (`.github/workflows/ci.yml:53-78`, `tests/regression/run-regression.sh`)
- HTTPS-only release tarball fetch in `install.sh:52-67`

Aspirational / Not Yet Implemented:

- **Sigstore provenance on npm publish.** Not yet wired. Should run `npm publish --provenance` on a GitHub Actions OIDC-authenticated workflow so consumers can `npm install` and verify Fulcio-issued provenance via `npm audit signatures`. Currently provenance is “trust GitHub releases + the maintainer’s npm token didn’t leak.”
- **Cosign-signed release artifacts.** `emit-evidence --sign` ships, but the *repo’s own release tarballs* are unsigned. `install.sh:52-67` fetches the release tarball over HTTPS without verifying anything.
- **Rekor entries for tagged releases.** Same as above — no transparency-log evidence that the v1.0.1 tarball is the one the maintainer published.
- **DNSSEC + CAA on evals.intentsolutions.io.** This is the precondition for the first Rekor push from `emit-evidence`. CISO binding from ISEDC Session 1 (`scripts/emit-evidence.sh:39-43`) — operator discipline, not script-enforced.
- **SBOM emission.** No `audit-harness sbom` or comparable. Adopters cannot generate a CycloneDX/SPDX manifest from the harness’s view of their repo.
- **Self-pinning.** `.harness-hash` for this repo’s own scripts is not initialized — `exit-3` (“no manifest”) is tolerated in CI (`.github/workflows/ci.yml:42-46`). The harness does not yet protect itself from tampering.
- **Bus-factor mitigation.** Sole maintainer; no secondary publish path or escrow.

10. Cost & Performance

Monthly Costs

The package is OSS and runs in adopters’ CI / pre-commit environments. Direct costs to Intent Solutions are operational only:

Table 9.

Resource	Cost	Notes
npm public registry	\$0	Free for public packages
PyPI hosting	\$0	Free for public packages
crates.io hosting	\$0	Free for public packages
GitHub Actions minutes	covered by GitHub Pro	CI workflow: ~3-5 minutes per PR across 3 Node versions + shellcheck + python-syntax + regression = ~15 runner-min per PR. Public-repo runner minutes are free.
Sigstore (Fulcio + Rekor)	\$0	Free public infrastructure for OSS
GitHub releases hosting	\$0	Tarball serving included
security@intentsolutions.io	covered by Workspace	Email channel
Maintainer time	sole-maintainer effort	Best-effort per SUPPORT.md

Cost-per-consumer-run: trivial. The harness executes in seconds (sub-second for **verify**, ~1-2s for **escape-scan** on a typical diff, ~10-60s for **crap** depending on project size and language scorer). On a typical CI runner this is well under \$0.01 per PR per consumer.

Performance

The harness is not a service — there is no P50/P95/P99 latency in the API sense. Per-invocation timings (rough, no formal benchmarks):

- **audit-harness --version**: <100ms (Node startup dominates)
- **audit-harness list**: <200ms
- **audit-harness verify**: O(N) in pinned-file count; <1s for typical repos (~10-30 files)
- **audit-harness escape-scan --staged**: O(M) in diff size; ~500ms for a 100-line diff; ~3-5s for 1000+ line diffs. Bash-grep-per-line dominates; the regex set is small.
- **audit-harness arch**: dominated by the consumer's checker (dependency-cruiser ~5-15s, ArchUnit / Gradle ~30-60s).
- **audit-harness crap**: dominated by the consumer's complexity scorer (radon ~2-5s; gocyclo + go test -cover ~30-90s for full coverage run).
- **audit-harness emit-evidence (unsigned)**: <500ms.
- **audit-harness emit-evidence --sign --rekor-url**: 2-8s (cosign OIDC handshake + Rekor PUT).

Throughput: bound by adopter CI parallelism, not the harness. No global state, no shared resources.

Error budget: not formally defined. Implicit goal — zero false-negative REFUSEs in 100% of weeks (a missed REFUSE means an AI-tampered diff merged, which is catastrophic). False positive REFUSEs are tolerable up to ~1 per consumer per week before adopters get fatigued and start `--no-verify-bypassing`.

Scaling Limits

- **Diff size:** `escape-scan.sh` reads the entire diff into bash variables via `grep -E '^\[^\+]' "$DIFF_SRC". For diffs over ~50K lines, bash variable handling becomes slow and may exceed ARG_MAX if interpolated. Mitigation: chunk diffs or use --range against a tighter range.`
- **Pinned-file count:** `harness-hash.sh:74-83` runs `sha256sum` per file in a `while read` loop. For 1000+ pinned files this is slow (~5-10s). No mitigation today; would need parallelization (`xargs -P` or GNU parallel).
- **CRAP scoring on large repos:** `crap-score.py` runs language-native scorers across the entire `src/` tree. For repos with 100K+ files this takes minutes. Mitigation: `--target src` (skip tests), `--lang <one>` (skip multi-language detection), or run async in CI.
- **Concurrent consumers:** not a constraint — there is no shared backend.
- **Predicate URI durability:** the URI is frozen post-Rekor-push (`SEMVER.md:73-77`). If the URI body shape needs breaking changes, `gate-result/v2` must coexist with `v1` in `emit-evidence`, doubling the body-marshaling code path until `v1` is sunset.

11. Current State

What's Working

- All eight gates implemented and exercised by CI on Node 18 / 20 / 22 (`.github/workflows/ci.yml:21-22`).
- Backward-compat regression suite (`tests/regression/run-regression.sh`) gates every PR with 4 sections / 11 checks.
- Three publish ecosystems wired and shipped (npm v1.0.1, PyPI v0.1.0, crates.io v0.1.0) plus `curl-pipe-bash install.sh` for everything else.
- Apache 2.0 licensing with NOTICE shipped in tarball (the v1.0.1 fix).
- SemVer commitment doc (`SEMVER.md`) codifies the consumer-promise contract.
- CODEOWNERS protection on `/scripts/`, `/bin/`, CI workflows, security-sensitive files.
- Dependabot weekly bumps for GitHub Actions + npm.
- Issue and PR templates require version + repro + Consumer-Repo-Impact disclosure.
- Phase A foundation deliverables present: `000-docs/001-DR-DESIGN-...` (Evidence Bundle envelope design) + `000-docs/002-RR-LAND-...` (upgrade landscape).
- Convergence-layer Evidence Bundle emission implemented and tested (`emit-evidence.sh` + Section 4 of regression suite).
- Zero runtime dependencies in the canonical Node package.

What Needs Attention

- **[HIGH]** Self-pinning gap — this repo does not yet initialize a `.harness-hash` against its own scripts. CI tolerates exit-3 on the self-check (`.github/workflows/ci.yml:36-51`). Impact: the harness does not protect itself from tampering. Fix: `audit-harness init` against this repo's `scripts/*` and `bin/*` and commit `.harness-hash`; tighten CI to fail on non-zero from `verify` and `list` once stable.

- **[HIGH]** No sigstore provenance on npm publish. Impact: consumers cannot verify that the v1.0.1 tarball was published by the legitimate maintainer. Fix: wire `npm publish --provenance` in a GitHub Actions OIDC workflow per npm docs. Reference precedent: `intent-eval-core@0.1.0` (per `umbrella CLAUDE.md`) ships with sigstore provenance — copy that workflow.
- **[HIGH]** Version drift across `package.json` (1.0.1) / `version.txt` (0.2.0) / `python/pyproject.toml` (0.1.0) / `rust/Cargo.toml` (0.1.0). Impact: consumers see different versions per install path; license fields disagree (canonical is Apache 2.0; Python and Rust wrappers still say MIT). Fix: sync all four to `package.json`'s value as part of the release flow; relicense PyPI + crates.io packages to Apache 2.0 on next publish.
- **[MEDIUM]** Threat model in `SECURITY.md` mentions supply-chain signed releases as “planned” (`SECURITY.md:47`). Impact: aspirational claim with no shipped mitigation. Fix: either ship sigstore provenance (see HIGH above) or downgrade the language to be honest about current state.
- **[MEDIUM]** `install.sh` fetches release tarballs without checksum or signature verification (`install.sh:52-67`). Impact: MITM during install replaces the harness with a malicious version. Fix: ship a `CHECKSUMS.txt` per release; have `install.sh` verify before extraction.
- **[MEDIUM]** macOS bash 3.2 vs Linux bash 5 not documented; `mapfile`, `[[ -v ]]`, process-substitution + `set -u` combinations may behave subtly differently. Impact: developer-laptop weirdness. Fix: add `BASH_VERSIONINFO` preamble check to scripts that need bash 4+ or document `brew install bash` in setup.
- **[MEDIUM]** `harness-hash.sh:40-60` `PATTERNS` array is closed. Impact: consumers cannot register custom policy artifacts for pinning. Fix: support a `.harness-hash-patterns` file in the consumer repo; bump minor version when shipped.
- **[LOW]** Documentation drift: `CONTRIBUTING.md:12` says Python 3.10+, `python/pyproject.toml:8` says `>=3.8`, CI uses 3.12. Settle on one.
- **[LOW]** `SECURITY.md:8` “Supported Versions” table lists `latest (v0.1.x)` — out of date; should reflect v1.x.
- **[LOW]** `dependabot.yml` ecosystems do not include `pip` (for Python wrapper) or `cargo` (for Rust wrapper). Impact: dependency bumps in those wrappers are manual.
- **[LOW]** No SBOM emission. Adopters who need CycloneDX/SPDX for compliance must generate it externally.

Implementation Status

Table 10.

Component	Status	Evidence
Node CLI dispatcher	Implemented + tested	<code>bin/audit-harness.js</code> ; CI matrix on Node 18/20/22
<code>harness-hash.sh</code> (verify/init/list)	Implemented + tested	<code>scripts/harness-hash.sh</code> ; regression Section 1

Component	Status	Evidence
<code>escape-scan.sh</code>	Implemented + tested	<code>scripts/escape-scan.sh</code> ; CI synthetic-input check at <code>workflows/ci.yml:53-78</code> ; regression Sections 1, 2
<code>arch-check.sh</code>	Implemented; not in regression suite	<code>scripts/arch-check.sh</code> ; dispatch-only — depends on consumer's installed tools
<code>bias-count.sh</code>	Implemented; advisory only	<code>scripts/bias-count.sh</code> ; never FAILs (exit 0 always per <code>SEMVER.md:50</code>)
<code>gherkin-lint.sh</code>	Implemented; awk fallback when official linter absent	<code>scripts/gherkin-lint.sh</code>
<code>crap-score.py</code>	Implemented for Python, Go, JS/TS, Rust	<code>scripts/crap-score.py:281-286</code> DISPATCH table
<code>emit-evidence.sh</code> (unsigned)	Implemented + tested	<code>scripts/emit-evidence.sh</code> ; regression Section 4
<code>emit-evidence.sh</code> (signed)	Implemented; blocked on DNSSEC + CAA	<code>scripts/emit-evidence.sh:39-43</code>
Python wrapper (<code>intent-audit-harness</code>)	Shipped at v0.1.0; out of sync	<code>python/pyproject.toml</code>
Rust wrapper (<code>intent-audit-harness</code>)	Shipped at v0.1.0; out of sync	<code>rust/Cargo.toml</code>
<code>install.sh</code> for non-Node	Implemented; no signature verification	<code>install.sh:52-67</code>
Backward-compat regression suite	Implemented + CI-gated	<code>tests/regression/run-regression.sh</code> ; CI <code>workflows/ci.yml:103-126</code>
Self-pinning manifest	NOT YET (gap)	CI <code>workflows/ci.yml:42-46</code> comment; HIGH finding
npm publish provenance	NOT YET (gap)	No <code>release.yml</code> in workflows (removed per commit <code>d5cef4e</code>); HIGH finding
OTel collector wiring	Stub only (stderr line emission)	<code>scripts/emit-evidence.sh:182-187</code>
Mutation-testing dispatcher	Deliberately not built	“What Was Deliberately Not Built” §4
SAST / SBOM / secret-scan dispatch	Deliberately not built	§4

12. Roadmap

Week 1 — Stabilization

Measurable outcomes:

- Initialize `.harness-hash` against this repo's own scripts (`scripts/*.sh`, `scripts/*.py`, `bin/audit-harness.js`). Commit. Tighten CI to fail on non-zero from `verify` and `list` instead of `|| true`. (Addresses §11 HIGH “Self-pinning gap”; resolves §8.10.)
- Sync versions: write a one-shot `scripts/sync-versions.sh` that propagates `package.json#version` to `version.txt`, `python/pyproject.toml`, `python/src/intent_audit_harness/__init__.py`, and `rust/Cargo.toml`. Run on the next release. (Addresses §11 HIGH “Version drift”.)
- Relicense PyPI + crates.io wrappers to Apache 2.0 on the next publish (currently both list `license = "MIT"` in their manifests despite the canonical relicense in v1.0.0). Publish hotfix wrapper releases.

Month 1 — Foundation

- Ship sigstore provenance on npm publish via a GitHub Actions OIDC workflow. Pattern: clone the release workflow from `intent-eval-core` per umbrella `CLAUDE.md`. (Addresses §11 HIGH “No provenance”; satisfies §9 aspirational item.)
- Add checksum + signature verification to `install.sh`. Ship `CHECKSUMS.txt` per release; verify in `install.sh` before tarball extraction. (Addresses §11 MEDIUM “install.sh no verification”; resolves §8.2.)
- Add `BASH_VERSIONINFO` preamble to scripts requiring bash 4+. Document `brew install bash` in `CONTRIBUTING.md` for macOS. (Resolves §8.7.)
- Add dependabot ecosystems for `pip` and `cargo` to `.github/dependabot.yml`. (Resolves §11 LOW “Dependabot ecosystems incomplete”.)
- Settle Python version floor: pick one of 3.8 / 3.10 / 3.12 and reconcile `pyproject.toml`, `CONTRIBUTING.md`, and CI. (Resolves §11 LOW “Python version drift”.)

Quarter 1 — Strategic

- Self-pinning extension mechanism: support `.harness-hash-patterns` in consumer repos so they can declare custom policy artifacts without forking. Minor version bump. (Resolves §8.4.)
- DNSSEC + CAA enablement on `evals.intentsolutions.io`. Coordinate with ISED CISO binding (`scripts/emit-evidence.sh:39-43`). Once verified, unblock the first Rekor push from `emit-evidence --sign --rekor-url`.
- Mutation-testing layer: add `audit-harness mutation` that dispatches Stryker / PIT / mut-mut / cargo-mutants. Per `000-docs/002-RR-LAND-...`, gate on incremental (changed-code-only) mode to manage performance.
- Bus-factor mitigation: document an escrow / co-maintainer plan; consider rotating an npm publish secret into the IEP umbrella GitHub Actions secrets so a release can be cut from a `workflow_dispatch` run rather than the maintainer's laptop. (Addresses §9 honest-assessment “Bus-factor mitigation”.)
- Stream-split clarity: prominent README documentation of `--json` semantics. Optional `--quiet` flag to suppress the stderr summary in `--json` mode for adopters who want JSON-only.

- L2 dispatch consideration: re-evaluate whether `audit-harness secret-scan` or `audit-harness sast` belongs in the surface. Currently deliberately out-of-scope (§4 “Not Built”), but downstream Evidence Bundle composition may force the issue.

13. Quick Reference

URLs

Table 11.

Resource	URL
npm package	https://www.npmjs.com/package/@intentsolutions/audit-harness
PyPI package	https://pypi.org/project/intent-audit-harness/
crates.io package	https://crates.io/crates/intent-audit-harness
GitHub repo	https://github.com/jeremylongshore/audit-harness
Issues	https://github.com/jeremylongshore/audit-harness/issues
Security email	security@intentsolutions.io
Sole maintainer email	jeremy@intentsolutions.io
Predicate URI (frozen post-Rekor)	https://evals.intentsolutions.io/gate-result/v1
Convergence umbrella issue	https://github.com/jeremylongshore/intent-eval-lab/issues/4
Evidence Bundle SPEC (canonical)	https://github.com/jeremylongshore/intent-eval-lab/blob/main/specs/evidence-bundle/v0.1.0-draft/SPEC.md
Cosign installation	https://docs.sigstore.dev/cosign/installation/
in-toto Statement v1	https://github.com/in-toto/attestation/blob/main/spec/v1/statement

First-Week Checklist

- ☐ Cloned the repo; `npm install` runs clean
 - ☐ `node bin/audit-harness.js --version` prints 1.0.1
 - ☐ Ran `bash tests/regression/run-regression.sh` and got `PASS=11 FAIL=0` (with `json-schema` installed)
 - ☐ Read `README.md`, `CLAUDE.md`, `SEMVER.md`, this document, `000-docs/001-DR-DESIGN-...`, `000-docs/002-RR-LAND-...`
 - ☐ Understood the `REFUSE` / `CHALLENGE` / `FLAG` grammar in `scripts/escape-scan.sh:95-103`
 - ☐ Understood the predicate-URI freeze rule in `SEMVER.md:69-77`
 - ☐ Walked through the critical path in §3 above end-to-end on a real diff
 - ☐ Reviewed the load-bearing files in §5 (`escape-scan`, `harness-hash`, `emit-evidence`, the dispatcher, `SEMVER`, `regression`, CI workflow)
 - ☐ Reviewed the “Things That Will Bite You” §8 before touching any pinned file or CI workflow
 - ☐ Confirmed access: GitHub permissions on the repo; `npm org-scope` token in `~/.npmrc` (maintainer only); PyPI + `crates.io` credentials (maintainer only)
 - ☐ Joined the `bd` workspace at `~/000-projects/.beads/` and ran `bd ready` to see in-flight work
 - ☐ Met with system owner (Jeremy Longshore, sole maintainer)
-

Appendices

A. Glossary

- **CHALLENGE** — `escape-scan` severity level; exit code 1; requires engineer-approved comment to merge. See `scripts/escape-scan.sh:9-12`.
- **CRAP** — Change Risk Analyzer and Predictor; $CRAP(m) = C(m)^2 * (1 - cov(m)/100)^3 + C(m)$. Industry-original metric (Savoia 2007, no peer-reviewed primary citation). See `scripts/crap-score.py:39-41` and `000-docs/002-RR-LAND-...` §3.
- **DSSE** — Dead Simple Signing Envelope; the in-toto envelope format that cosign produces when signing a Statement. See `scripts/emit-evidence.sh:219-249`.
- **Evidence Bundle** — the upstream schema for gate-result rows authored in `intent-eval-lab/specs/evidence`. The harness emits one row per gate-invocation via `emit-evidence`.
- **Fulcio** — sigstore’s certificate authority that issues short-lived X.509 certs via OIDC for keyless signing.
- **gate-result** — the JSON envelope shape emitted by every gate’s `--json` mode. Becomes the predicate body when wrapped by `emit-evidence`.
- **gate-result/v1** — the frozen predicate URI path; `https://evals.intentsolutions.io/gate-result/v1`. See `SEMVER.md:69-77`.
- **HARNESS_TAMPERED** — `harness-hash --verify` exit code 2; pinned file changed without a fresh `--init`. See `scripts/harness-hash.sh:9` and `scripts/escape-scan.sh:147-155`.
- **in-toto Statement v1** — open-standard attestation envelope; `_type`: `https://in-toto.io/Statement/v1`. See `scripts/emit-evidence.sh:158-166`.
- **ISED** — Intent Solutions Executive Decision Council; 7-seat adversarial council. The CISO binding referenced in `scripts/emit-evidence.sh:39-43` came from ISED Session 1 (2026-05-10).
- **Rekor** — sigstore’s public transparency log; immutable record of signed Statements.
- **REFUSE** — `escape-scan` severity level; exit code 2; halts the pipeline. The hardest gate.

- **schema_version** — semver-versioned envelope contract for gate-result rows. See 000-docs/001-DR-DESIGN-... §“Versioning rules”.
- **stream split** — the v0.3.0 `--json` convention: `stdout = JSON`, `stderr = human-readable`. See `SEMVER.md:60-65`.
- **7-layer testing taxonomy** — the meta-framework the harness lives inside. L1 (git hooks / CI), L2 (static / SAST), L3 (unit + coverage + mutation + arch + CRAP — *audit-harness lives here*), L4 (integration / contract), L5 (perf / sec / ally / chaos), L6 (E2E / BDD), L7 (acceptance / RTM / personas / journeys). See `README.md:98-110`.

B. Reference Links

- Anthropic Claude Code skills spec: <https://docs.anthropic.com/en/docs/claude-code/skills>
- Intent Solutions Testing SOP: umbrella `~/000-projects/CLAUDE.md` § “Intent Solutions Testing SOP”
- Intent Eval Platform umbrella: `~/000-projects/intent-eval-platform/CLAUDE.md`
- Evidence Bundle SPEC: <https://github.com/jeremylongshore/intent-eval-lab/blob/main/specs/evidence-bundle/v0.1.0-draft/SPEC.md>
- Sigstore docs: <https://docs.sigstore.dev/>
- Keep a Changelog: <https://keepachangelog.com/en/1.1.0/>
- SemVer: <https://semver.org/spec/v2.0.0.html>
- in-toto attestation framework: <https://github.com/in-toto/attestation>
- SLSA provenance v1.0: <https://slsa.dev/spec/v1.0/>

C. Troubleshooting Playbooks

Playbook: a consumer reports “the harness let an obvious threshold-lowering diff through”

1. Reproduce locally with the offending diff: `bash scripts/escape-scan.sh --no-hash <patch-file>`. Expect exit 2.
2. If exit 0, the gate is broken — file P0.
3. Inspect `tests/TESTING.md` in the consumer repo — is the floor parser silent-falling-back? Check the format matches `escape-scan.sh:82-89`.
4. Inspect `.harness-hash` — is it present? If not, `--no-hash` is implicit; the in-band hash check is skipped.
5. Inspect the patch — does it have a comment chain that pattern-defeats the regex? Add a regression case to `tests/fixtures/`.
6. Patch the regex; verify against the existing fixtures; add the new fixture; ship a patch release with a SECURITY-tagged CHANGELOG entry.

Playbook: audit-harness verify exits 2 but the engineer believes no pinned file changed

1. `audit-harness list` — what’s pinned?
2. `bash scripts/harness-hash.sh --verify` — exit 2 message includes the diff between manifest and current hashes (`harness-hash.sh:134-135`).
3. Inspect each line of the diff; the changed file’s path is named.
4. If the change is intentional: `audit-harness init`; commit both the policy file and `.harness-hash` together.
5. If the change is *not* intentional: someone (or some tool) modified a pinned file behind the engineer’s back. Investigate: `git log -p <file>`, `git blame`, check for CI-job-side mutations.

Playbook: emit-evidence --sign --rekor-url fails

1. `cosign version` — installed?
2. If using `--keyless`: do you have a valid OIDC identity? `cosign sign` against a test artifact first to confirm the OIDC handshake works.
3. If using `--key cosign.key`: is the key path readable? Does it match the public key registered in Rekor?
4. Rekor push 5xx: check sigstore status page; retry with backoff. The script exits 3 with a clear message if Rekor push fails (`scripts/emit-evidence.sh:249-252`).
5. If pushing against a custom Rekor URL: confirm the URL accepts uploads and that DNSSEC + CAA on the *predicate* namespace `evals.intentsolutions.io` is verified (per CISO binding, `scripts/emit-evidence.sh:39-43`).

Playbook: regression suite fails on Section 1 (text-mode parity)

This is the SemVer-critical fail mode. It means the existing text output has drifted.

1. `bash tests/regression/run-regression.sh` and read the specific FAIL line.
2. Diff the actual vs expected text. The expected string is the v0.2.0 baseline — what the package *promised* adopters in `SEMVER.md`.
3. If the drift is *intentional*, the change requires a MAJOR version bump per `SEMVER.md:20`. Stop the PR; discuss with the maintainer.
4. If the drift is unintentional, revert the offending change in the gate script.

D. Open Questions

These are unresolved as of v1.0.1 and should be tracked as discussion items, not action items (yet):

1. **Should escape-scan warn when running without .harness-hash?** Currently silent — the gate runs but skips the in-band hash check. A warning to stderr would help adopters notice the missing layer.
2. **Should the harness emit a schema_version field on every gate's --json output?** Currently the envelope shape is 1.0 by convention but not explicit. Adding the field is forward-compatible (per 000-docs/001-DR-DESIGN-... §“Versioning rules”) and helps downstream consumers reject unknown majors.
3. **Should bias-count produce a non-zero exit code when grade is CRITICAL?** Currently always exit 0 (advisory). `SEMVER.md:50` codifies “always 0 (advisory gate)” — changing it is a MAJOR bump. Worth doing or worth holding?
4. **Is the OTel stderr-line emission the right place to stop?** `scripts/emit-evidence.sh:182-187` deliberately stops at “structured signal on stderr.” Is that enough for the convergence-layer Evidence Bundle consumers, or do they want a real OTel SDK push?
5. **Should the predicate URI be filed as a registered in-toto predicate type?** SCAI (Melara 2022, 000-docs/002-RR-LAND-... §3 #5) is registered; `gate-result/v1` is currently private to Intent Solutions. Registration is a one-way door — once filed, breaking changes are *very* expensive.
6. **Should there be a gate-result/v1.1 minor-bump path documented?** Per `SEMVER.md:69-77` the URI body is frozen, but the design doc (000-docs/001-DR-DESIGN-... §“Versioning rules”) describes minor bumps for additive optional fields. Reconcile.
7. **Bus-factor: should publish secrets be escrowed in a GHA-secrets vault so the cohort can cut a release?** Per umbrella `CLAUDE.md`, the Anthropic Enterprise cohort

has 35 subcontractors; one of them could in principle act as a release deputy if the secrets were available outside the maintainer's laptop.